



1 8 0 3

INFORMÁTICA II

Bioingeniería - Ingeniería Biomédica



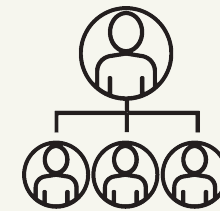
Índice



Repaso



Abstracción



Herencia



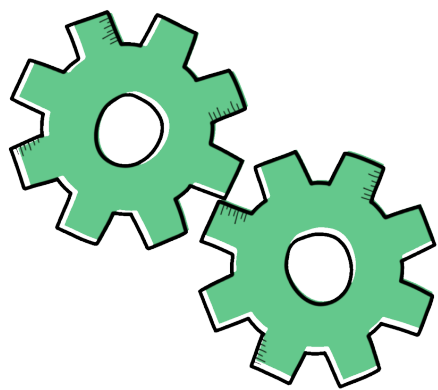
Encapsulamiento



Notación UML



Reunir conceptos





Repaso de conceptos de la clase pasada:

- En qué consiste la POO? 
- Qué ventajas otorga el spyder , Visual Studio?  
- Cuál es la diferencia entre ejecutar y depurar?  
- Para qué sirve esta materia? 



P
O
O



Abstracción

Es la capacidad de aislar, extraer las características y comportamientos mas generales que describen un objeto cotidiano.

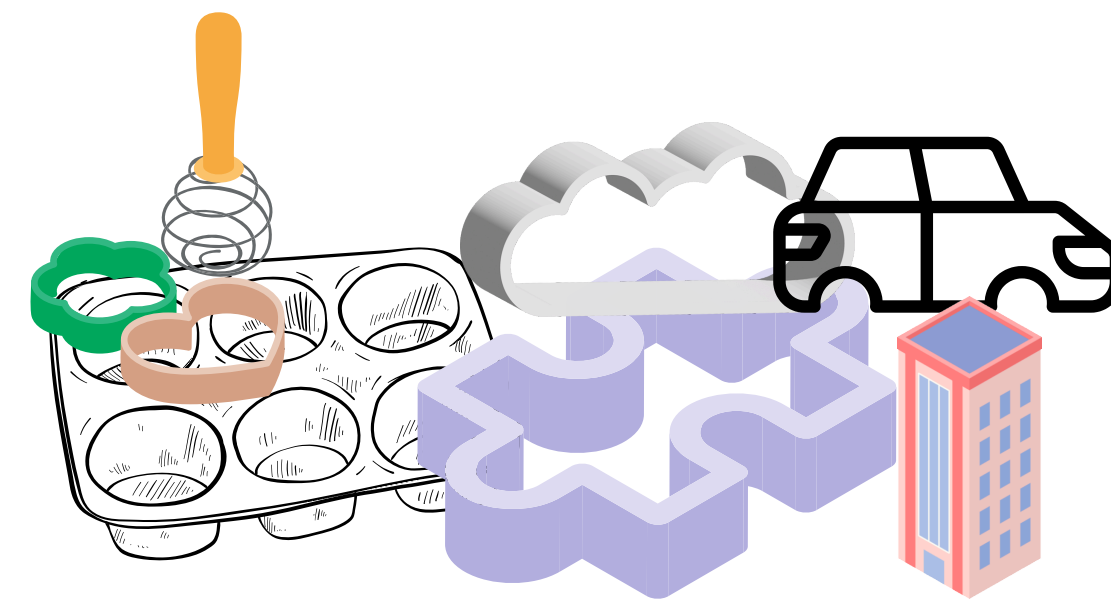
Clase:

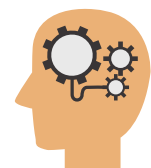
Conjunto de atributos y métodos relacionados a un elemento relevante del problema a resolver.

Sirve como **PLANTILLA** para construir objetos

Objeto:

Variable en memoria construida a partir de una plantilla , patrón ó **CLASE**





Abstracción



Clase:

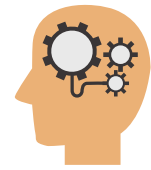
Plantilla que se define con la palabra class en python.

Objeto:

Variable (*identificador = Tipo()*) en memoria construida a partir de una clase

```
class Carro:  
    pass
```

```
a = Carro()  
b = Carro()
```



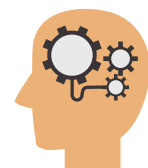
Abstracción

En python todo , absolutamente todo es un objeto

¿Cómo usamos los objetos y clases en python entonces ?

```
1  a = 1
2  b = ["Luis", 10,10203456]
3  c = {"primero":b,2:{2,3,4},3:"Hola Mundo"}
4  d = (2,3,4,4)
5  e = "hola"
6  d = list
7  f = 0.2398
```





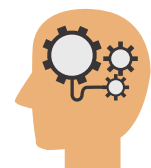
Abstracción



Método `__init__`, constructor:

Todas las clases tienen un método `__init__` (con doble guión bajo) el cual es llamado de manera automática cuando la clase es inicializada.

La palabra **self**, permite identificar el propio objeto que está ejecutandose, de entre todos los que pueden existir en memoria en un momento dado



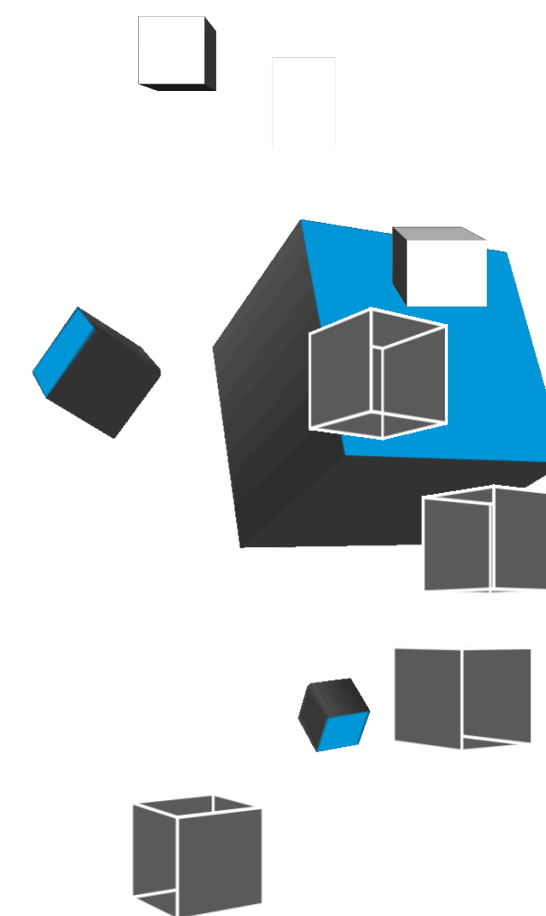
Abstracción



Método `__init__`, constructor:

Todas las clases tienen un método `__init__` (con doble guión bajo) el cual es llamado de manera automática cuando la clase es inicializada.

La palabra **self**, permite identificar el propio objeto que está ejecutándose, de entre todos los que pueden existir en memoria en un momento dado. Permite identificar los atributos y metodos del objeto actual, esto último através del operador punto ('.')



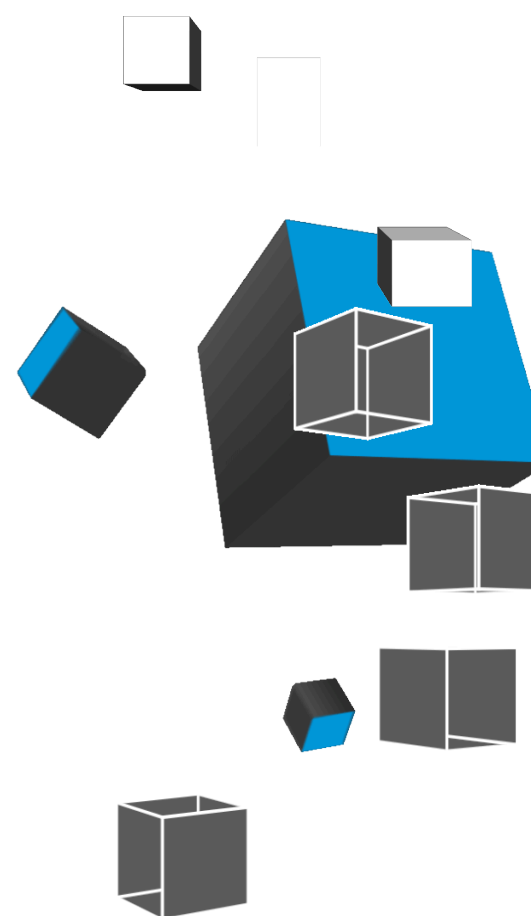


Abstracción

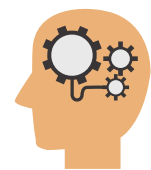


Ejercicios:

- Cómo haríamos una **clase** persona y otra coche?
- Cuáles serían sus **atributos y métodos**?
- Cómo creamos un **objeto** de esta clase?
- Cómo imprimimos en consola un atributo de un **objeto**?



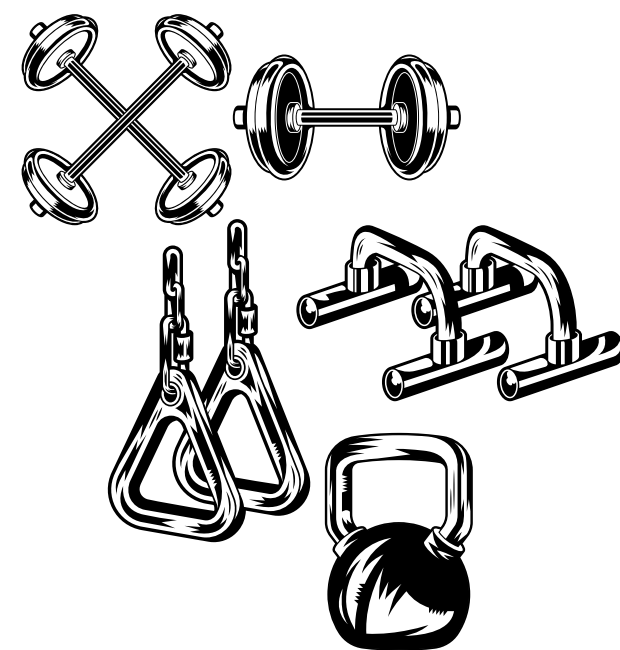
https://www.w3schools.com/python/python_classes.asp

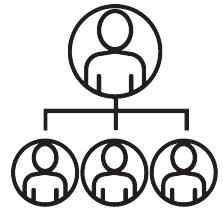


Abstracción

Ejercicios:

- Qué tienen en común estos objetos?

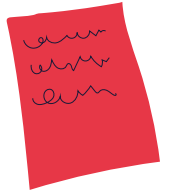
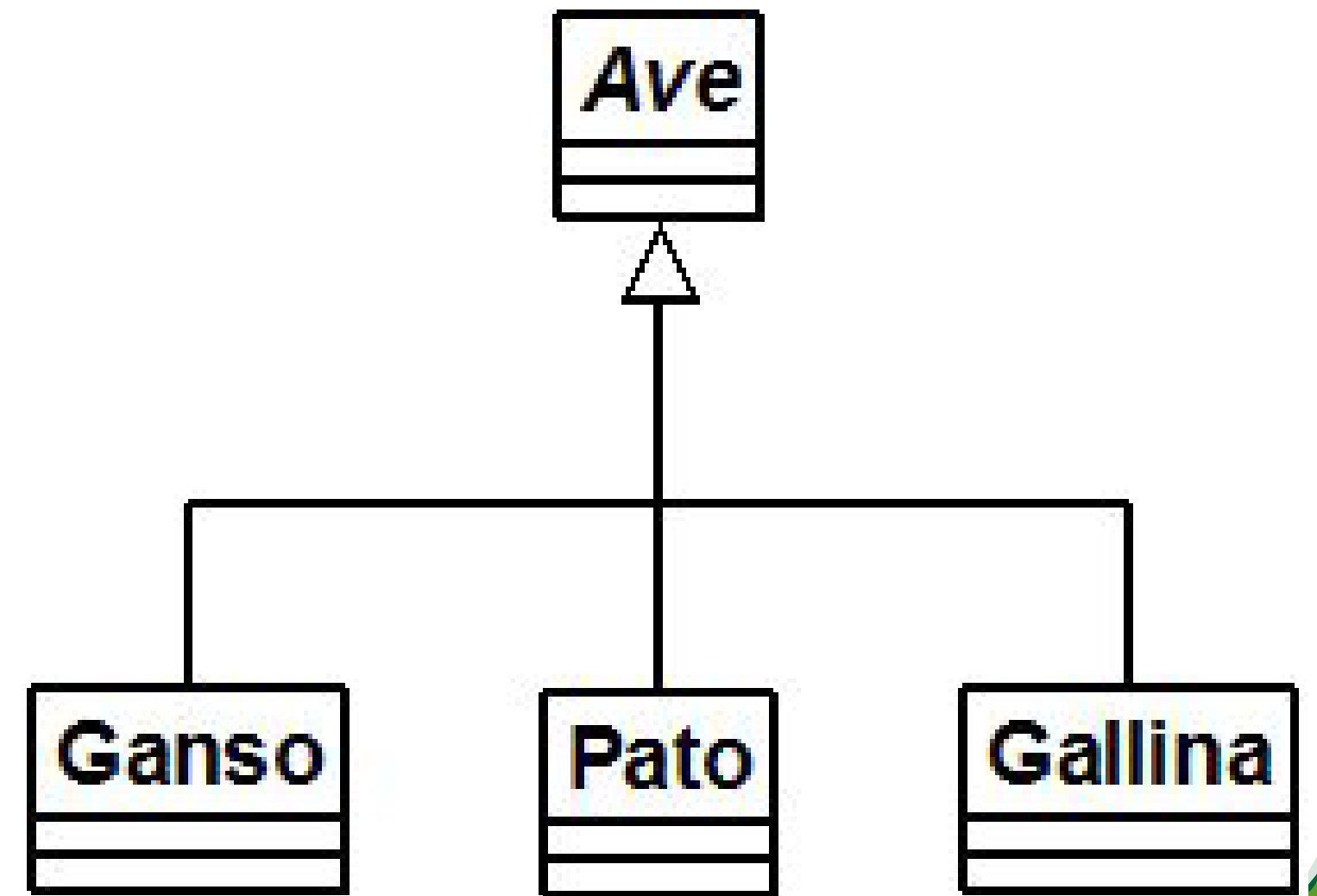


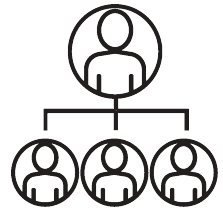


Herencia

Permite definir relaciones de generalización donde se parte , de lo general a lo concreto

Ayuda a la reutilización de código y a la adaptación de funcionalidades nuevas mediante el **polimorfismo**





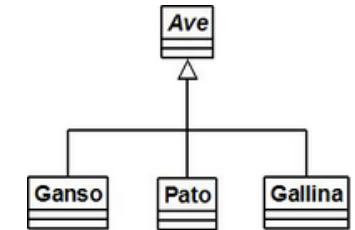
Herencia



```
class ave:
    # Constructor de ave en su forma mas abstracta o básica
    def __init__(self, tipo, vuela):
        self.ave = tipo # referente si es carnívora, insectívora, omnívora, carroñera, etc
        self.vuelo = vuela
        self.oviparos = True
        self.pico = True

    # Acciones básicas
    def comer(self, comida):
        print("Este tipo de ave come normalmente : ", comida)

    def volar(self):
        print("Este tipo de ave puede volar: ", self.vuelo)
```



```
class ganso(ave):

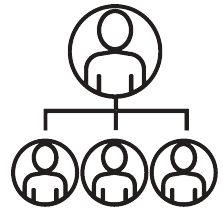
    def __init__(self, tipo, vuela, accion, pata):
        # Invoca al constructor de clase ave
        ave.__init__(self, tipo, vuela)
        # Nuevos atributos
        self.habilidad = accion # Puede volar y/o nadar
        self.patas = pata

    def destreza(self):
        print("Esta ave puede : ", self.habilidad)

class pato(ganso):
    pass

class gallina(ave):
    pass
```





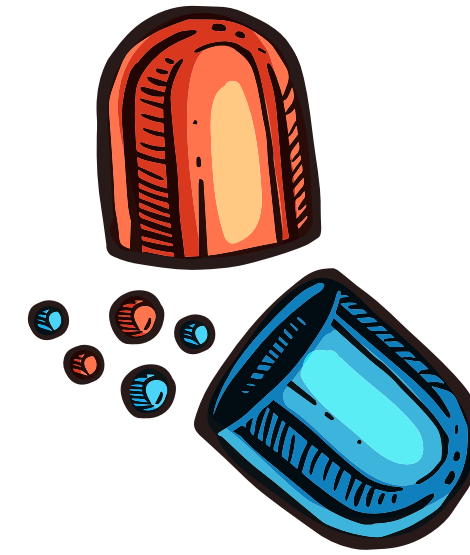
Encapsulamiento

En principio los atributos no se deberían poder manipular por fuera de la clase, aplicable también a los métodos: Es decir, hacer que los atributos y/o métodos internos de una clase sean solo manipulables desde dentro de los instancias de dichas clases.

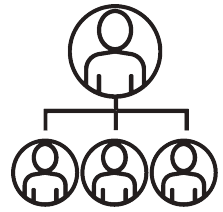
Para evitar esto, y hacer atributos sean privados, se usa

self.__nombreVariable

self.__nombreFuncion



```
12 class Paciente:
13     def __init__(self):
14         self.nombre = "Luis"
15         self.edad = 10
16         self.cedula = 10203456
17
18 j = Paciente()
19 print(j.nombre)
20 print(j.edad)
```



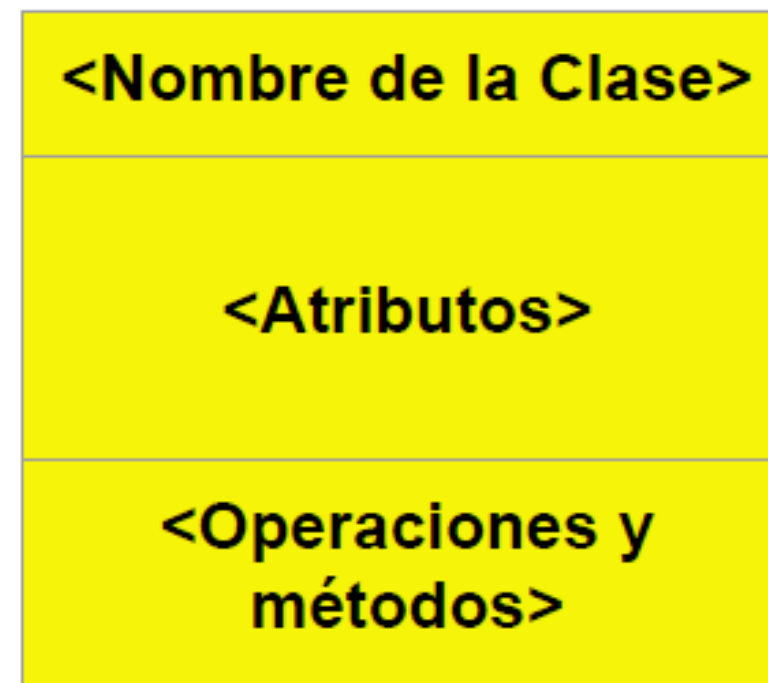
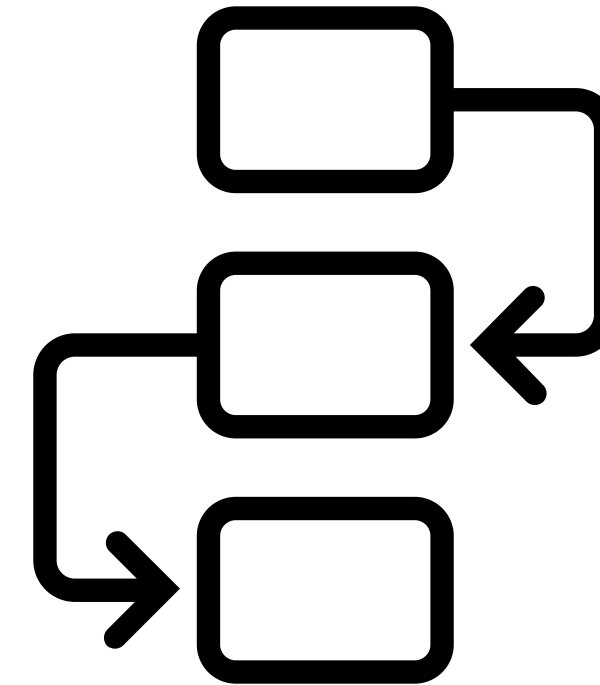
Notación UML

Para representar gráficamente los componentes y relaciones entre clases se usa el **Lenguaje Unificado de Modelado**

Nombre de clases: Sustantivos, singular (**Perro, Casa, Estudiante**), inician en mayúscula

Nombre de métodos: Verbos (**asignar, inicializar**), inician en minúscula, si son palabras compuestas, la segunda iniciará en mayúscula (**asignarNombre**)

Nombre de atributos: Sustantivos (**horas,color, altura, listaEstudiantes**)

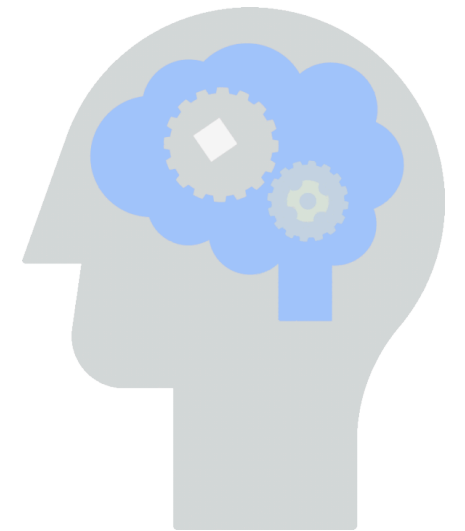




Reunir conceptos



- Hagamos un repaso de las palabras clave aprendidas hoy y su relación con la POO.
- Hagamos un ejercicio de abstracción, si quiero alquilar casas, cómo modelaría la casa usando POO?
- Repasemos el uso de self, construyamos una clase y dos objetos, Python, cómo diferencia los atributos un objeto del otro?
- Corregir la parte de encapsulamiento de los códigos
- Usar el método id para ver la posición en memoria de diferentes objetos creados a partir de una clase definida
- Crear una clase Estudiante y otra EstudianteBioingenieria usando herencia



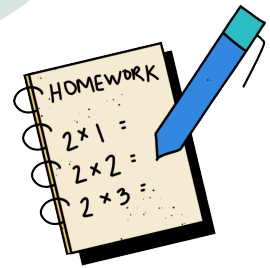


Para consultar y hacer



- Cómo se implementa la herencia múltiple en Python?
- Qué es el polimorfismo?
- Cómo se usa el polimorfismo en python?
- Crear dos clases donde una componga la otra ,
ejemplo: clase Computador y clase microchip, el computador esta hecho con microchips





Para adelantar en casa....



Primer ejercicio de abstracción:

Se solicita construir un sistema que permita almacenar datos de pacientes, médicos y enfermeras.

Los pacientes tienen información de nombre, cédula, género y servicio, que corresponde al servicio donde están alojados.

Las enfermeras tienen la misma información que los pacientes, menos servicio, mas turno (que puede ser 7-19, 19-7, corrido) y rango (auxiliar, jefe, jefe del servicio).

Adicionalmente pueden existir médicos que tienen toda la información de las enfermeras, menos el rango, más especialidad





...gracias